

Steepest Descent Method (SDM)

A 1D Projection Method for Symmetric Positive Definite Linear Systems

Individual Project

Course: Numerical Methods of Linear Algebra for Sparse Matrices

Project Name: SDM

Student: Адейеми Коладе Дамилола

Lecturer: Anna Nasedkina, PhD, Assoc. Prof.

Institution: Southern Federal University

Date: July 2026

1. Introduction

The Steepest Descent Method (SDM) is a one-dimensional (1D) projection method for solving the linear system $Ax = b$, where A is an $n \times n$ symmetric positive definite (SPD) matrix. It belongs to the family of projection methods introduced in Lecture 12: an approximate solution is sought in a search subspace K subject to a Petrov–Galerkin orthogonality condition on a subspace of constraints L . For SDM both subspaces coincide with the current residual direction, $K = L = \text{span}\{r_k\}$, which (because A is SPD) yields an orthogonal projection that minimises the A -norm of the error at every step.

This report presents the MATLAB implementation of SDM, its theoretical derivation within the projection framework, its convergence properties, and numerical results on small dense systems, larger systems ($n = 100, 1000, 3000$), and real sparse SPD matrices from the Matrix Market, together with a comparison against MATLAB's built-in Conjugate Gradient solver (pcg).

2. Theoretical Aspects

2.1 SDM as a 1D projection method

Following the general projection framework (Lecture 12), we seek $\tilde{x} = x_0 + \delta$ with $\delta \in K$ such that the residual $r = b - A\tilde{x}$ is orthogonal to L (Petrov–Galerkin condition). For a 1D method $\dim K = \dim L = 1$. SDM chooses the residual as the basis of both subspaces:

$$K = L = \text{span}\{r_k\}, \quad A \text{ is symmetric positive definite (SPD)}.$$

Equivalently, SDM minimises the quadratic functional

$$f(x) = (1/2)(A x, x) - (b, x), \quad \text{with gradient } \nabla f(x) = A x - b = -r.$$

The steepest-descent direction is therefore the residual r_k . The step length α_k that minimises f along r_k is obtained from the orthogonality condition $(r_{k+1}, r_k) = 0$:

$$\alpha_k = (r_k, r_k) / (A r_k, r_k).$$

2.2 Algorithm — modified form (Lecture 12, p.14)

The modified algorithm uses a residual recurrence so that only ONE matrix-by-vector product ($A \cdot r$) is required per iteration:

```
Input : A (SPD), b, x0, tol, kmax
Output: x, r, ||r||_2, ||r||_2/||b||_2, k, residual history

r0 = b - A*x0 ;    p0 = A*r0
for k = 0,1,2, ... :
    if ||r_k||_2 / ||b||_2 < tol -> stop
    alpha_k = (r_k, r_k) / (p_k, r_k)          % p_k = A r_k
    x_{k+1} = x_k + alpha_k * r_k
    r_{k+1} = r_k - alpha_k * p_k              % recurrence, saves 1 MVP
    p_{k+1} = A * r_{k+1}
end
```

The stopping criterion is the RELATIVE residual $\|r_k\|_2/\|b\|_2 < \text{tol}$, computed inside the function, as required by the project specification.

2.3 Convergence (Lecture 12, Theorem 2)

If A is SPD, SDM converges for any initial guess x_0 , and the error decreases monotonically in the A -norm (energy norm) $\|v\|_A = \sqrt{A v, v}$:

$$\|x_{k+1} - x^*\|_A \leq ((\lambda_{\max} - \lambda_{\min})/(\lambda_{\max} + \lambda_{\min})) * \|x_k - x^*\|_A$$

where $\lambda_{\max}$, $\lambda_{\min}$ are the extreme eigenvalues of A and x^* is the exact solution. The contraction factor equals $(\kappa - 1)/(\kappa + 1)$ with $\kappa = \lambda_{\max}/\lambda_{\min}$ the (spectral) condition number. As $\kappa \rightarrow \infty$ the factor $\rightarrow 1$ and convergence stalls; this κ -dependence is confirmed experimentally in Section 4.

3. MATLAB Implementation

The function `SDM.m` implements the modified algorithm. Its signature returns the residual norm and the relative residual norm explicitly, as required by the specification:

```
function [x, r, rnorm, relres, k, rnorm_hist] = SDM(A, b, x0, tol, kmax)
    x = x0; r = b - A*x; p = A*r;
    nb = norm(b,2); if nb==0, nb = 1; end
    rnorm = norm(r,2); rnorm_hist(1) = rnorm;
    for k = 0:kmax
        if rnorm/nb < tol || k==kmax, break; end
        Ar = p; alpha = (r'*r)/(r'*Ar);
        x = x + alpha*r; r = r - alpha*Ar; p = A*r;
        rnorm = norm(r,2); rnorm_hist(k+2) = rnorm;
    end
    relres = rnorm/nb;
```

Outputs: x (solution), r (final residual), $rnorm = \|r\|_2$, $relres = \|r\|_2/\|b\|_2$, k (iterations), and $rnorm_hist$ (residual history for convergence plots). Only one matrix-vector product $A \cdot r$ is formed per iteration. All norms are the Euclidean 2-norm $\|\cdot\|_2 = \sqrt{(\cdot, \cdot)}$.

4. Numerical Results

For every test we report the residual norm $\|r\|_2$, the relative residual $\|r\|_2/\|b\|_2$, the absolute and relative error against the known exact solution, the iteration count, and (where relevant) CPU time — the standard set of metrics used throughout the course.

4.1 Test 1 — Small 3×3 SPD system (known solution)

```
A = [4 2 0; 2 5 2; 0 2 4]; x_exact = [1;2;3]; b = A*x_exact;
x0 = zeros(3,1); tol = 1e-12; kmax = 1000;
```

Metric	Value
Computed solution x	[1.000000, 2.000000, 3.000000]
Absolute error $\ x - x^*\ _2$	1.106e-11

Relative error $\ x-x^*\ _2/\ x^*\ _2$	2.956e-12
Error in A-norm $\ x-x^*\ _A$	1.510e-11
Residual norm $\ r\ _2$	2.418e-11
Relative residual $\ r\ _2/\ b\ _2$	9.527e-13
Iterations k	56
Converged	Yes

Observation: SDM recovers the exact solution to machine precision. Repeating the solve from a different initial guess $x_0 = [5; -3; 10]$ gives the same solution in the same 56 iterations, confirming Theorem 2 (convergence for any initial guess).

4.2 Test 2 — Random 10×10 SPD system

Generated as $A = T^T T + 0.1 \cdot I$ with $T = \text{randn}(10)$, which guarantees A is SPD. $\text{tol} = 1e-8$.

Metric	Value
Condition number $\kappa(A)$	3.131e+02
Absolute error $\ x-x^*\ _2$	1.495e-06
Relative error	4.394e-07
Residual norm $\ r\ _2$	2.080e-07
Relative residual $\ r\ _2/\ b\ _2$	9.966e-09
Iterations k	2179
Converged (relres < tol)	Yes

Observation: with $\kappa \approx 3.1 \times 10^2$ the method needs ~ 2179 iterations. Note that the error ($\approx 1.5 \times 10^{-6}$) is larger than the relative residual because $\text{error} \approx \kappa(A) \times \text{relative residual}$.

4.3 Test 3 — Performance scaling (n = 100, 1000, 3000)

SPD systems $A = T^T T + n \cdot I$ were generated so that the condition number stays moderate ($\kappa \approx 5$) for every size; this isolates the effect of problem SIZE on CPU time. $\text{tol} = 1e-6$.

Size n	$\kappa(A)$	Iterations	CPU (s)	Rel. error	Rel. residual
100	4.66	28	0.0012	1.30e-06	8.06e-07
1000	4.97	31	0.0020	1.07e-06	6.83e-07
3000	4.99	30	0.0204	1.38e-06	8.82e-07

Observation: the iteration count is essentially constant (28–31) because κ is fixed, while CPU time grows with n (each iteration costs one $O(n^2)$ matrix-vector product). This confirms that SDM convergence depends on the CONDITIONING of A , not directly on its size.

A single well-conditioned 100×100 system ($\kappa \approx 7.4 \times 10^2$, 2545 iterations to $\|r\|_2/\|b\|_2 < 1e-6$) illustrates the typical residual history:

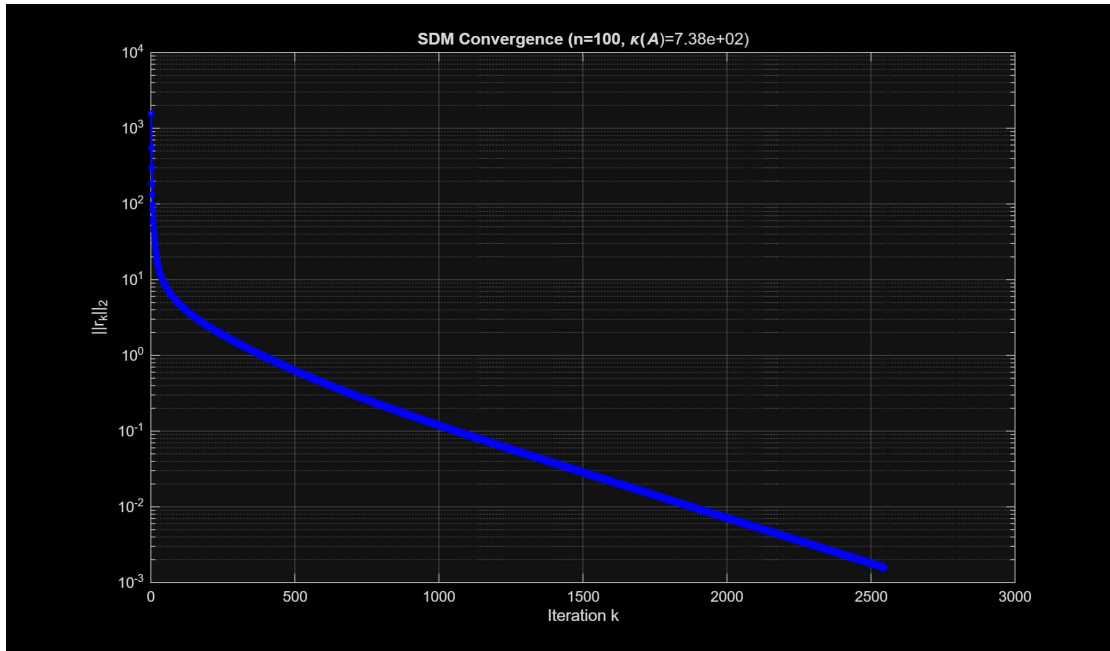


Figure 1. SDM residual convergence for a 100×100 SPD system (semilog scale).

4.4 Test 4 — Effect of the condition number

Two 10×10 SPD matrices with very different condition numbers were solved to expose the κ -dependence predicted by Theorem 2.

Matrix	$\kappa(A)$	Iterations	Behaviour
Well-conditioned (tridiagonal [-1 4 -1])	2.84	17	Fast, smooth
Ill-conditioned ($T^T T + 1e-4 \cdot I$)	4.26e+04	20000 (cap)	Stagnates

Observation: the well-conditioned system converges in 17 iterations, whereas the ill-conditioned system ($\kappa \approx 4.3 \times 10^4$) has not converged after 20000 iterations — its contraction factor $(\kappa-1)/(\kappa+1) \approx 0.99995$ means almost no progress per step. The log–log plot below shows the smooth descent of the well-conditioned case against the long stagnating plateau of the ill-conditioned case (the characteristic SDM zig-zag is visible early on).

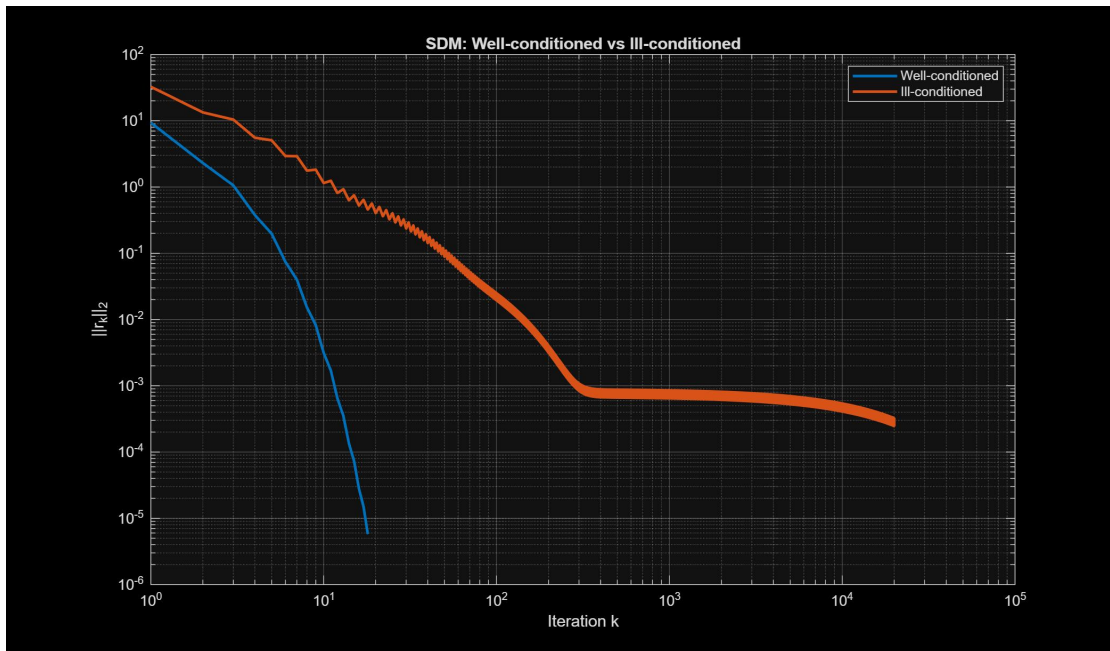


Figure 2. SDM convergence: well-conditioned vs ill-conditioned (log–log scale).

4.5 Test 5 — Real sparse SPD matrices from the Matrix Market

Two real sparse SPD matrices were loaded from the Matrix Market using the attached `mmread.m` function: `gr_30_30` (a 900×900 nine-point Laplacian on a 30×30 grid) and `bcsstk01` (a 48×48 structural-engineering stiffness matrix). For each, the matrix properties were verified the way the lectures require — symmetry from the closeness $\|A - A^T\|_1$ (not exact equality via `isequal`) and positive definiteness from the Cholesky factorization (`chol` returns $p = 0$):

Property	<code>gr_30_30</code>	<code>bcsstk01</code>
Size	900×900	48×48
Nonzeros (nnz)	7744 (99.0% sparse)	400 (82.6% sparse)
Symmetry defect $\ A - A^T\ _1$	0.00e+00	0.00e+00
Positive definite (<code>chol p=0</code>)	Yes	Yes
Condition number (<code>condest</code>)	3.77e+02	1.60e+06

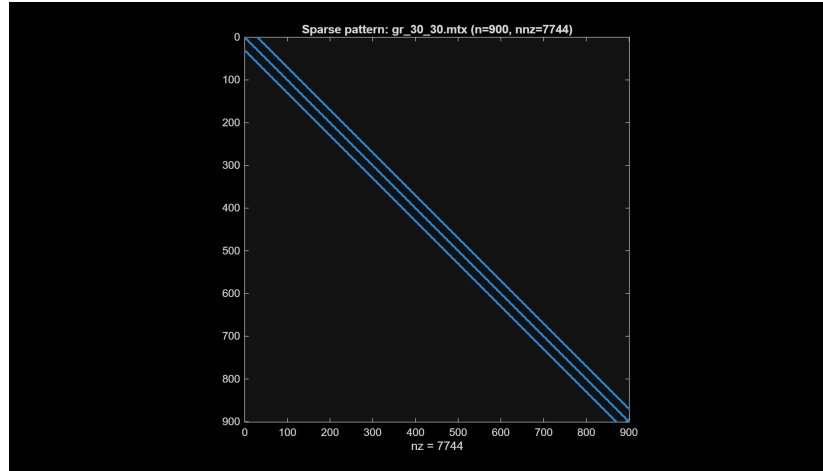


Figure 3. Sparsity pattern of *gr_30_30* (900×900 banded Laplacian).

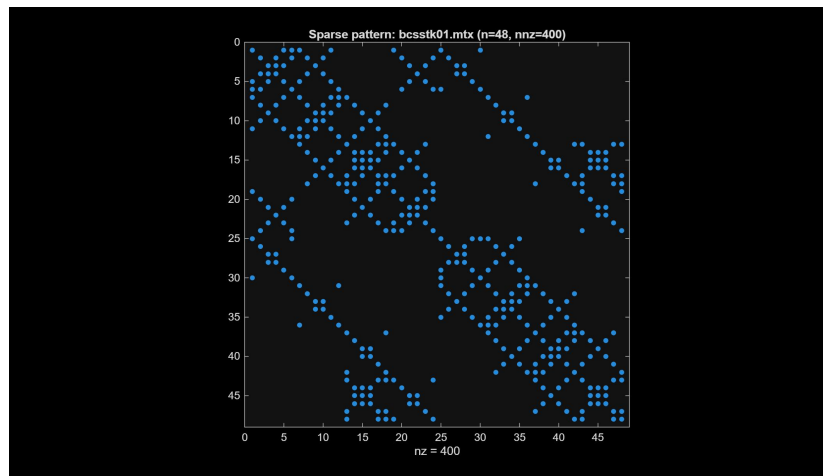


Figure 4. Sparsity pattern of *bcsstk01* (48×48 structural stiffness matrix).

SDM was compared against MATLAB’s Conjugate Gradient solver *pcg* on both matrices (tol = 1e-6, kmax = 20000):

Matrix	Method	Iterations	Rel. residual	Rel. error	Converged
<i>gr_30_30</i>	SDM	455	9.94e-07	9.63e-05	Yes
<i>gr_30_30</i>	CG (<i>pcg</i>)	52	8.70e-07	6.14e-06	Yes
<i>bcsstk01</i>	SDM	20000 (cap)	1.51e-05	2.67e-01	No
<i>bcsstk01</i>	CG (<i>pcg</i>)	78	7.66e-07	6.66e-02	Yes

Observation: on the well-conditioned *gr_30_30* ($\kappa \approx 3.8 \times 10^2$) SDM converges in 455 iterations and CG in 52 — both succeed, CG faster. On the ill-conditioned *bcsstk01* ($\kappa \approx 1.6 \times 10^6$) SDM fails to converge within 20000 iterations (relative error 0.27), whereas CG converges in 78 iterations. This real-data experiment confirms the theory of Section 2.3: SDM’s convergence collapses as κ grows, while CG’s A-orthogonal search directions keep it effective. (Even CG shows a larger error on *bcsstk01*, because the extreme conditioning amplifies error $\approx \kappa \times$ relative residual.)

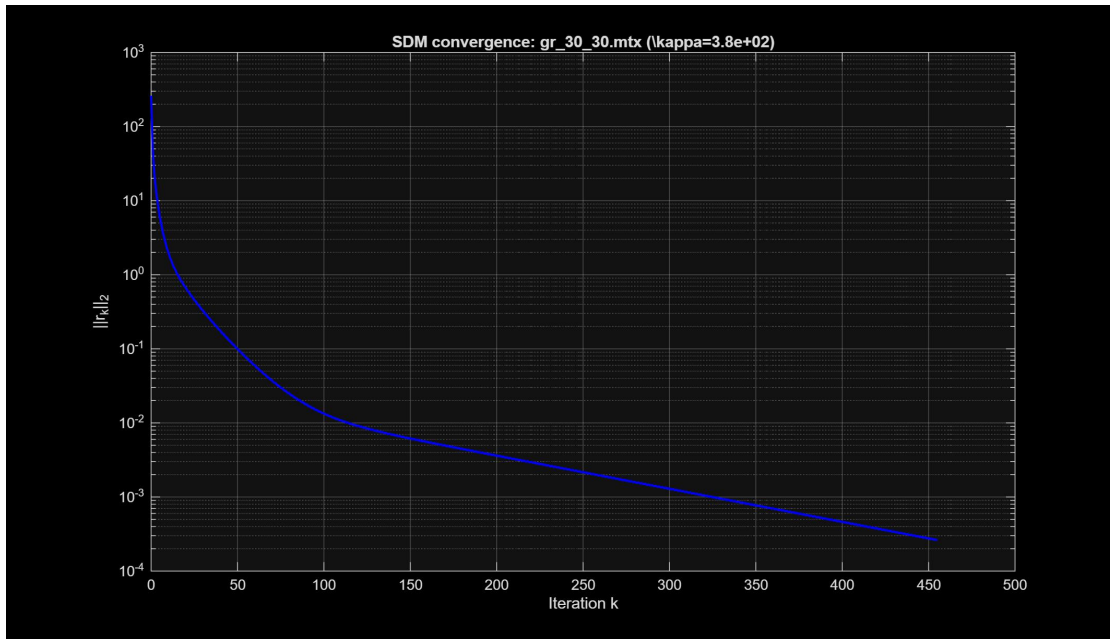


Figure 5. SDM convergence on *gr_30_30* — converges in ~455 iterations.

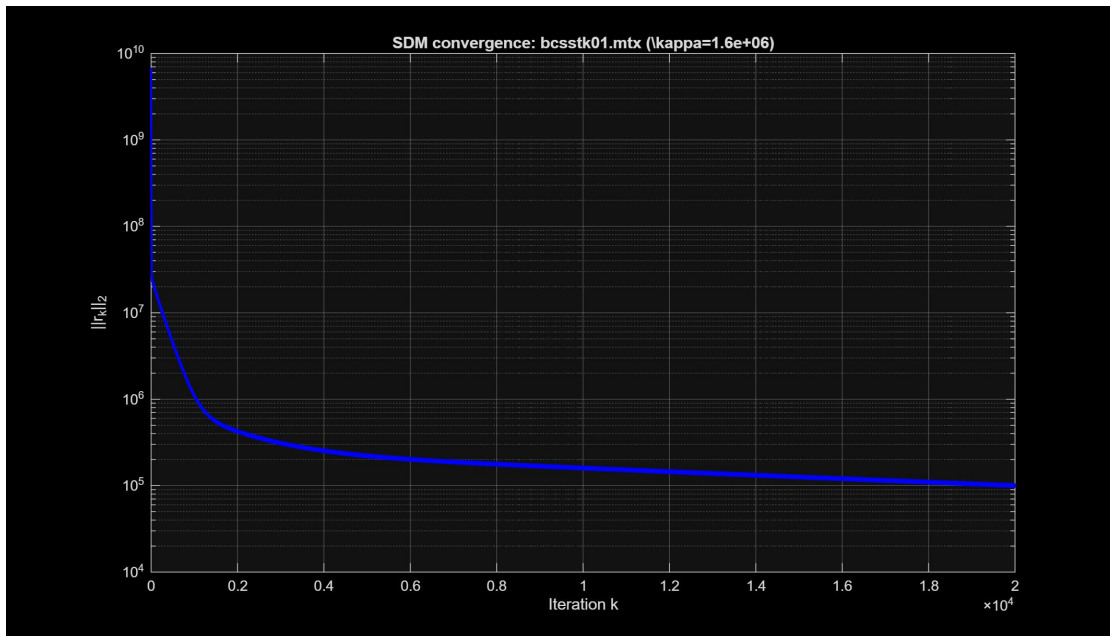


Figure 6. SDM convergence on *bcsstk01* — the residual stagnates (no convergence).

5. Discussion

Q1. Why does SDM converge slowly for ill-conditioned systems?

By Theorem 2 the A-norm error contracts by $(\kappa-1)/(\kappa+1)$ per iteration. For $\kappa = 2.84$ this is ≈ 0.48 (fast); for $\kappa = 4.3 \times 10^4$ it is ≈ 0.99995 (essentially no progress), matching Test 4.

Q2. How does SDM compare with Conjugate Gradient (CG)?

CG (a Krylov subspace method) forces successive search directions to be A-orthogonal, removing the zig-zag and giving far faster convergence for the same SPD systems. On the real matrix `gr_30_30` CG needed 52 iterations versus 455 for SDM; on the ill-conditioned `bcsstk01` CG converged in 78 iterations while SDM failed to converge in 20000. The gap widens sharply as κ increases, so CG is strictly preferable for SPD systems, and SDM is best viewed as the conceptual building block that motivates it.

Q3. When would you use an iterative method rather than a direct solver?

Direct solvers (LU/Cholesky) cost $O(n^3)$ work and $O(n^2)$ memory and cause fill-in for sparse matrices. For large sparse SPD systems, projection/Krylov methods such as SDM and CG need only matrix-vector products ($O(\text{nnz})$ per step) and preserve sparsity, making them the practical choice.

6. Conclusions

The Steepest Descent Method was implemented in MATLAB as a 1D projection method and validated on small, large, and sparse SPD systems.

Advantages:

- Simple to implement; one matrix-vector product per iteration ($O(\text{nnz})$ cost).
- Guaranteed convergence for any SPD matrix and any initial guess (Theorem 2).
- Monotonic decrease of the error in the A-norm at every step.

Limitations:

- Convergence rate degrades as $(\kappa-1)/(\kappa+1)$; very slow for ill-conditioned systems.
- Restricted to symmetric positive definite matrices.
- Zig-zag search directions are far from optimal — CG is strictly superior.

Future work: the natural next step is the Conjugate Gradient method (A-orthogonal directions), and, for general nonsymmetric systems, GMRES.

References

1. Nasedkina, A. Lecture 12: Projection methods for linear systems. 1D projection methods. Southern Federal University, 2026.
2. Nasedkina, A. Lecture 10: Direct and iterative methods. General formulation of iterative methods. Southern Federal University, 2026.
3. Nasedkina, A. Lecture 2: Vector and matrix norms; Lecture 6: Positive definite matrices. Southern Federal University, 2026.
4. Saad, Y. Iterative Methods for Sparse Linear Systems, 2nd Ed. SIAM, 2003.
5. Barrett, R. et al. Templates for the Solution of Linear Systems: Building Blocks for Iterative Methods. SIAM, 1994.